Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Build ASP.NET Web Apps with Vibe Coding

TGS Ref No: TGS-2023040477

Software Design · ICT-DES-3005-1.1

Conducted by

Tertiary Infotech Academy Pte Ltd

Version 6.1 · 12 September 2026

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
6.1	12 September 2026	Modernised ASP.NET Core 10 courseware with ten cumulative LocalCRM Activities, full step-by-step procedures, accessibility, API, SQLite, Azure-readiness and traceable design evidence.	Dr. Alfred Ang

TABLE OF CONTENTS

How to Use This Guide

Course Learning Outcomes

Before You Start

Topic 1 Design Components and the ASP.NET Core Runtime

Topic 2 Accessible UI and Organisational Vibe Coding Practice

Topic 3 Razor Pages Controls Binding and Persistence

Topic 4 Controller APIs and Component Interoperability

Topic 5 Production Hosting Azure and Design Evidence

Quick Command Reference

Assessment Flow

Support

How to Use This Guide

This guide provides the detailed, step-by-step procedures intentionally omitted from the visual trainer deck. Complete Activities 1–10 in sequence. Each Activity includes its own runnable src/LocalCRM snapshot and fictional mock data, so you can recover from an earlier failed change without copying from another folder.

Use an approved AI coding assistant only with mock data. A vibe script is a bounded design input; it is never proof that the code is correct. Inspect the diff, build, test, probe both positive and negative paths, and record evidence.

Course Learning Outcomes

- LO1: Determine basic software components using ASP.NET framework to meet functional specifications.
- LO2: Utilize ASP.NET framework to develop web apps in adherence to organizational practices.
- LO3: Select essential ASP.NET controls and features to meet software design requirements.
- LO4: Examine the interoperability and functionality of ASP.NET software components.
- LO5: Generate ASP.NET design documentation aligned with user specifications.

Before You Start

You need a computer with a .NET 10 SDK, Git, a modern browser, and a code editor. Node.js is optional for the axe accessibility CLI. Activities 8–10 restore EF Core SQLite packages from NuGet. Azure credentials are not required: the deployment work is designed and verified locally.

```
dotnet --info
git --version
dotnet new list web
```

Expected evidence: a 10.0 SDK is listed, Git responds, and the webapp template is available.

Topic 1: Design Components and the ASP.NET Core Runtime

Alignment: LO1 · K1 / A1

requirements → component boundaries → request pipeline → verified scaffold

HTTP request and response contract

Mechanism

1. Browser — creates method, URL and headers

2. Kestrel — accepts the TCP/TLS request

3. Middleware — applies cross-cutting policy

4. Endpoint — binds input and runs logic

5. Response — status, headers and body return

Exact contract

- Method: GET reads; POST creates; PUT replaces; DELETE removes
- Target: /contacts/42 plus query string
- Headers: Accept, Content-Type, Authorization, trace context
- Body: JSON or form payload; never implied by a GET
- Status: 2xx success, 4xx caller issue, 5xx server failure

```
app.MapGet("/health", () => Results.Ok(new { status = "ok" }));
```

Failure and verification

- 404: route template did not match
- 405: path matched but method did not
- 415: Content-Type cannot be read
- 500: unhandled endpoint exception
- Verify: curl -i /health returns HTTP 200
- Verify: Content-Type is application/json
- Verify: The response body matches the declared contract

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

ASP.NET technology family boundary

Mechanism

1. Classic ASP — .asp + VBScript on IIS

2. ASP.NET 4.x — Web Forms/MVC 5 on .NET Framework

3. ASP.NET Core — cross-platform unified web stack

4. App model — Razor Pages, MVC, API, Blazor

5. Host — Kestrel directly or behind proxy

Exact contract

- Classic ASP: Server-side script; legacy maintenance only
- Web Forms: .aspx event lifecycle; not part of ASP.NET Core
- MVC 5: System.Web on .NET Framework
- ASP.NET Core MVC: Microsoft.AspNetCore.* packages
- Target: net10.0 for this course

```
<TargetFramework>net10.0</TargetFramework>
<Nullable>enable</Nullable>
```

Failure and verification

- Wrong tutorial: System.Web examples do not compile on Core
- Wrong file: .aspx controls are not Razor
- Wrong runtime: .NET Framework is Windows-only
- Wrong host: IIS concepts are mistaken for the app pipeline
- Verify: dotnet --list-sdks includes 10.0
- Verify: csproj targets net10.0
- Verify: No System.Web dependency exists

Source basis: Microsoft Learn — ASP.NET tutorials + ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

SDK, runtime and target framework

Mechanism

1. global.json — selects an allowed SDK

2. dotnet CLI — restores and builds project

3. Roslyn — compiles C# into IL

4. .NET runtime — loads assemblies and JITs code

5. ASP.NET host — starts Kestrel and endpoints

Exact contract

- SDK: build tools plus templates
- Runtime: executes a built application
- TFM: net10.0 API compatibility contract
- RID: optional platform-specific publish target
- Lock: global.json prevents silent SDK drift

```
{
  "sdk": { "version": "10.0.100", "rollForward": "latestPatch" }
}
```

Failure and verification

- NETSDK1045: installed SDK cannot target net10.0
- Restore error: package source or version is unavailable
- Works on mine: SDK version was not pinned
- Runtime missing: framework-dependent publish reached an empty host
- Verify: dotnet --info records SDK and runtime
- Verify: dotnet build ends with 0 errors
- Verify: global.json is committed with the design

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Project template and file ownership

Mechanism

1. dotnet new — materialises a reviewed template

2. csproj — declares framework and packages

3. Program.cs — owns composition root

4. wwwroot/Pages — own static and Razor UI

5. obj/bin — generated outputs stay disposable

Exact contract

- webapp: Razor Pages project template
- webapi: HTTP API template
- --no-https: only for constrained local tests
- --use-controllers: selects controller API style
- --dry-run: previews files before creation

```
dotnet new webapp -n LocalCRM --framework net10.0
dotnet new list web
```

Failure and verification

- Template absent: required SDK/workload is missing
- Generated files edited: obj/bin were mistaken for source
- Secret committed: local settings were treated as portable
- Name collision: namespace and folder identity diverged
- Verify: dotnet new --dry-run shows the intended template
- Verify: git status excludes bin/ and obj/
- Verify: dotnet run exposes the generated endpoint

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Composition root and dependency injection

Mechanism

1. CreateBuilder — collects services and configuration

2. Services — register abstractions and lifetimes

3. Build — freezes the service provider

4. Request scope — resolves scoped dependencies

5. Dispose — releases scoped resources

Exact contract

- Singleton: one instance for process lifetime
- Scoped: one instance per HTTP request
- Transient: new instance per resolution
- Constructor: declares required dependency
- Interface: separates consumer from implementation

```
builder.Services.AddScoped<IContactStore, SqliteContactStore>();
var app = builder.Build();
```

Failure and verification

- Cannot resolve: service was never registered
- Captive dependency: singleton holds a scoped service
- Service locator: runtime GetService hides the contract
- Duplicate registration: last registration silently wins
- Verify: Startup completes without DI exception
- Verify: One scoped instance is reused within a request
- Verify: Unit test can replace IContactStore

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Configuration provider precedence

Mechanism

1. appsettings.json — stores safe defaults

2. Environment JSON — overrides per environment

3. User secrets — protect local development secret

4. Environment vars — override deployment values

5. Options — bind and validate typed config

Exact contract

- Key path: ConnectionStrings:LocalCRM
- Env encoding: double underscore represents colon
- Secret: never committed to appsettings.json
- Options: ValidateOnStart catches invalid config
- Precedence: later provider overrides earlier

```
builder.Services.AddOptions<AppOptions>()
 .BindConfiguration("App")
 .ValidateDataAnnotations().ValidateOnStart();
```

Failure and verification

- Null option: key path does not match property
- Dev leak: secret landed in source control
- Wrong environment: ASPNETCORE_ENVIRONMENT is incorrect
- Late failure: options were not validated at startup
- Verify: Startup rejects an invalid required setting
- Verify: env var changes the resolved value
- Verify: secret scan finds no credential

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Middleware pipeline ordering

Mechanism

1. Exception handler — catches downstream failures

2. HTTPS/static — normalises transport and assets

3. Routing — selects endpoint candidates

4. AuthN/AuthZ — creates identity then enforces policy

5. Endpoint — executes selected handler

Exact contract

- Use: can run before and after next
- Run: terminal middleware
- Map: branches by path
- Order: registration order is execution order
- Short-circuit: response ends without downstream call

```
app.UseExceptionHandler("/Error");
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapRazorPages();
```

Failure and verification

- 401/403 surprise: authorization ran before authentication
- Headers absent: middleware ran after endpoint

- Static 404: static files were not mapped
- Details leak: developer exception page reached production
- Verify: Trace log shows expected middleware order
- Verify: Production errors use generic ProblemDetails
- Verify: Protected endpoint enforces its policy

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Vibe coding control loop

Mechanism

1. Specify — state outcome, files and constraints

2. Generate — AI proposes the smallest patch

3. Inspect — review diff and dependency changes

4. Verify — build, test and probe behaviour

5. Record — commit decision and evidence

Exact contract

- Context: target files and accepted architecture
- Constraints: net10.0, no secrets, accessible UI
- Acceptance: commands and exact observable result
- Diff budget: reject unrelated changes
- Rollback: checkpoint before generation

```
Prompt: Add GET /health to Program.cs only.
Constraints: net10.0; no new packages.
Verify: dotnet test && curl -i /health
```

Failure and verification

- Prompt drift: scope did not name files
- Hallucinated API: package/version was not checked
- Silent regression: only the happy path was probed
- Mega-diff: one prompt mixed unrelated changes
- Verify: git diff contains only declared files
- Verify: dotnet test is green
- Verify: manual probe produces named evidence

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Activity 1: Translate Requirements into a Verified ASP.NET Core Scaffold

Turn a LocalCRM customer-contact brief into component boundaries and a reproducible .NET 10 Razor Pages project.

Folder: activities/activity-01-requirements-and-scaffold

Vibe script

Act as a software-design assistant. Convert the supplied LocalCRM brief into UI, application-service, data and hosting components. Preserve requirement IDs. Do not write code yet. Return

assumptions separately.

Step-by-step

Step 1 — Create a clean workspace

Create the cumulative project folder used throughout all ten Activities.

```
mkdir -p localcrm && cd localcrm && git init
```

Step 2 — Record the functional brief

Create requirements.md with REQ-001 create a contact, REQ-002 list contacts, REQ-003 accessible form, and REQ-004 expose a JSON API.

Create or update requirements.md:

```
# LocalCRM requirements

- REQ-001: Staff can create a contact with name, email and lifecycle stage.
- REQ-002: Staff can list saved contacts.
- REQ-003: The form is keyboard-operable and exposes labels/errors.
- REQ-004: An HTTP JSON API exposes create, list and item retrieval.
```

Step 3 — Ask the AI for boundaries

Use the Activity prompt. Save the response in component-map.md; check that every component maps to at least one REQ ID.

Step 4 — Pin the SDK

Confirm the installed .NET 10 SDK, then create global.json.

```
dotnet --list-sdks
dotnet new globaljson --sdk-version 10.0.100 --roll-forward latestPatch
```

Step 5 — Preview the template

Inspect the files before materialising them.

```
dotnet new webapp -n LocalCRM --framework net10.0 --dry-run
```

Step 6 — Create and build

Create the Razor Pages app and compile it.

```
dotnet new webapp -n LocalCRM --framework net10.0
dotnet build LocalCRM/LocalCRM.csproj
```

Expected evidence: Build succeeded with 0 errors.

Step 7 — Run and probe

Start the app on a known local URL in one terminal; probe from another.

```
dotnet run --project LocalCRM --urls http://localhost:5050
curl -i http://localhost:5050/
```

Expected evidence: HTTP/1.1 200 and an HTML Content-Type header.

Step 8 — Checkpoint

Exclude generated outputs, inspect the diff and commit the verified scaffold.

```
printf 'bin/\nobj/\n*.db\n' > .gitignore
git status --short
git add requirements.md component-map.md global.json .gitignore LocalCRM
git commit -m 'feat: scaffold LocalCRM from requirements'
```

Run the supplied snapshot

```
cd activities/activity-01-requirements-and-scaffold/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5101
```

Activity 2: Run a Controlled Vibe-Coding Change

Add a health endpoint through a specify–generate–inspect–verify loop without accepting unrelated AI edits.

Folder: activities/activity-02-controlled-vibe-change

Vibe script

> Modify LocalCRM/Program.cs only. Add GET /health returning HTTP 200 JSON { status: 'ok', service: 'LocalCRM' }. Do not add packages or reorder existing middleware. Explain the exact diff and provide curl acceptance evidence.

Step-by-step

Step 1 — Confirm the baseline

Work from the Activity 1 commit and make sure the tree is clean.

```
git status --short
dotnet build LocalCRM/LocalCRM.csproj
```

Step 2 — Submit the constrained prompt

Use the provided prompt with your approved coding assistant. Reject any response that adds packages or edits more than Program.cs.

Step 3 — Apply the smallest patch

Place the endpoint after app.UseAuthorization() and before app.MapRazorPages().

Create or update Program.cs excerpt:

```
app.UseAuthorization();

app.MapGet("/health", () => Results.Ok(new
{
    status = "ok",
    service = "LocalCRM"
}));

app.MapRazorPages();
```

Step 4 — Inspect scope

Confirm exactly one source file changed and no package reference was added.

```
git diff --stat
git diff -- LocalCRM/Program.cs
git diff -- LocalCRM/LocalCRM.csproj
```

Step 5 — Build and run

Compile and start the application on the known URL.

```
dotnet build LocalCRM/LocalCRM.csproj
dotnet run --project LocalCRM --urls http://localhost:5050
```

Step 6 — Capture positive evidence

Probe headers and JSON from another terminal.

```
curl -i http://localhost:5050/health
```
Expected evidence: HTTP 200, application/json, status=ok and service=LocalCRM.

Step 7 — Capture negative evidence

Probe a misspelled path to prove routing distinguishes unmatched endpoints.

```
curl -i http://localhost:5050/healthz
```
Expected evidence: HTTP 404.

Step 8 — Record the decision

Create evidence/activity-02.md with prompt, reviewed files, commands and observed status codes; then commit.

```
mkdir -p evidence
git add LocalCRM/Program.cs evidence/activity-02.md
git commit -m 'feat: add verified health endpoint'
```

Run the supplied snapshot

```
cd activities/activity-02-controlled-vibe-change/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5102
```

Topic 2: Accessible UI and Organisational Vibe-Coding Practice

Alignment: LO2 · K2 / A2

semantic structure → accessible interaction → reviewable AI-assisted change

Semantic document structure

Mechanism

1. HTML parser — builds the DOM

2. Landmarks — identify page regions

3. Headings — form a navigable outline

4. Controls — expose native roles and states

5. AT — maps semantics into accessibility APIs

Exact contract

- main: one primary content region
- nav: named navigation collection
- h1–h6: do not skip levels for styling
- button: action with keyboard behaviour
- a[href]: navigation to a real target

```
<main>
  <h1>CRM contacts</h1>
  <button type="submit">Add contact</button>
</main>
```

Failure and verification

- Div soup: visual boxes have no programmatic role
- Heading jump: outline no longer communicates hierarchy
- Clickable span: keyboard activation is missing

- Duplicate main: landmark navigation becomes ambiguous
- Verify: Accessibility tree exposes main and button
- Verify: Heading outline has one h1
- Verify: Tab and Enter activate the control

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Accessible name computation

Mechanism

1. Element — starts with native semantics

2. aria-labelledby — references visible text first

3. aria-label — supplies explicit fallback

4. Content — native text may name control

5. Name — screen reader announces purpose

Exact contract

- label for: associates text with an input id
- alt: names an informative image
- empty alt: marks decorative image
- aria-describedby: adds help, not the name
- Unique id: prevents broken reference

```
<label for="email">Summary</label>
<input id="email" name="email" type="email" aria-describedby="email-help">
```

Failure and verification

- Placeholder only: name disappears after typing
- Filename announced: image has missing or poor alt
- Duplicate id: label binds to the wrong control
- ARIA override: visible and spoken names disagree
- Verify: Browser Accessibility pane shows the intended name
- Verify: Screen reader announces label and help
- Verify: axe reports no form-label violation

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Keyboard and focus contract

Mechanism

1. Tab — moves through interactive controls

2. Focus indicator — shows current target

3. Enter/Space — activates native control

4. Validation — moves attention to actionable error

5. Return — restores focus after transient UI

Exact contract

- Order: DOM order should match visual order
- tabindex=0: adds custom widget to natural order
- tabindex=-1: allows scripted focus only
- :focus-visible: keeps visible keyboard cue
- Escape: closes modal/menu and restores focus

```
button:focus-visible { outline: 3px solid #1f6feb; outline-offset: 3px; }
```

Failure and verification

- Focus trap: user cannot leave a component
- Focus loss: removed node held focus
- Positive tabindex: manual ordering becomes brittle
- Mouse-only handler: keyboard event path is absent
- Verify: Entire workflow completes without mouse
- Verify: Focus indicator remains visible
- Verify: Modal Escape returns focus to opener

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Contrast and non-colour cues

Mechanism

1. Tokens — define foreground/background pair

2. CSS cascade — resolves actual computed colours

3. Contrast ratio — compares relative luminance

4. State cue — adds text/icon/border beyond colour

5. Audit — checks each interactive state

Exact contract

- Normal text: target 4.5:1
- Large text: target 3:1
- UI component: target 3:1 against adjacent colour
- Link: not identified by colour alone
- State: hover, focus, disabled and error all checked

```
.error { color:#b42318; border-left:4px solid #b42318; }
.error::before { content:"Error: "; font-weight:700; }
```

Failure and verification

- Token passes only once: hover state drops contrast
- Red/green only: colour vision users lose status
- Text on image: background changes under glyphs
- Disabled too faint: essential text becomes unreadable
- Verify: Contrast checker records each token pair
- Verify: Status remains understandable in grayscale
- Verify: Lighthouse/axe result is attached

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Validation error experience

Mechanism

1. Input — user submits form

2. Model binding — converts named fields

3. Validation — adds errors to ModelState

4. Summary/field — renders actionable messages

5. Focus — moves to summary or first invalid field

Exact contract

- asp-validation-for: field-specific message
- asp-validation-summary: page-level error collection
- aria-invalid: marks failing input
- describedby: associates message
- Server check: remains authoritative without JavaScript

```
<span asp-validation-for="Input.Email"></span>
<div asp-validation-summary="ModelOnly" tabindex="-1"></div>
```

Failure and verification

- Client-only: bypassed request accepts invalid data
- Generic error: user cannot identify repair
- Focus unchanged: screen-reader user misses result
- Values lost: failed post rebuilds an empty form
- Verify: Empty submit shows specific message
- Verify: Error is associated with the field
- Verify: Server rejects the same invalid payload

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Lighthouse and axe evidence workflow

Mechanism

1. Build — run the reviewed application

2. Navigate — load a stable test state

3. Scan — execute Lighthouse and axe rules

4. Inspect — confirm each automated finding

5. Retest — record fixed score and residual manual checks

Exact contract

- Automated: finds detectable rule violations
- Manual: keyboard, reading order and meaning

- Baseline: store score and violation IDs
- Regression: fail build on new serious violations
- Evidence: HTML/JSON report plus screenshot

```
npx @axe-core/cli http://localhost:5000 --exit
# Expected: 0 violations (manual checks still required)
```

Failure and verification

- False confidence: score was treated as full conformance
- Flaky state: scan ran before page settled
- Third-party noise: scope was not documented
- No baseline: regression cannot be distinguished
- Verify: axe exits 0 for the target state
- Verify: Manual keyboard checklist is signed
- Verify: Report names URL, date and build SHA

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Prompt constraints for accessible output

Mechanism

1. Requirement — names user and task

2. Semantic constraint — requires native elements first

3. Interaction constraint — defines keyboard/focus behaviour

4. Evidence — names axe and manual acceptance

5. Review — rejects ARIA added without need

Exact contract

- Files: limit AI to the intended Razor/CSS files
- Standard: reference WCAG outcome, not vague accessibility
- Prohibition: no clickable divs or positive tabindex
- Acceptance: keyboard path plus automated scan
- Output: request patch and rationale, not prose only

```
Prompt: Make Pages/Contacts/Create.cshtml keyboard complete.
Use native controls; preserve label text.
Acceptance: axe=0 and Tab→Enter submits.
```

Failure and verification

- Vague prompt: model optimises appearance only
- ARIA cargo cult: redundant roles conflict with native semantics
- Hidden focus: CSS reset removes outline
- Untested claim: AI says accessible without evidence
- Verify: Diff uses native elements
- Verify: Tab order matches DOM order
- Verify: Acceptance evidence is reproducible

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Git diff and review guardrails

Mechanism

1. Checkpoint — commit or stash known-good state

2. AI patch — changes bounded files

3. Diff — inspect code, packages and deletions

4. Checks — format, build, tests and secret scan

5. Commit — record intent and evidence

Exact contract

- git diff --stat: detects unexpected breadth
- git diff --check: finds whitespace defects
- dotnet format --verify-no-changes: enforces style
- dotnet test: proves behavioural contract
- Secret scan: blocks credentials before history

```
git diff --check
dotnet format --verify-no-changes
dotnet test
git diff --name-only
```

Failure and verification

- Generated noise: bin/obj entered the diff
- Package surprise: AI added dependency without review
- Secret leak: connection string was committed
- Mixed commit: unrelated changes obscure review
- Verify: Only scoped source files changed
- Verify: All local gates exit 0
- Verify: Commit message links requirement and evidence

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Activity 3: Build an Accessible Razor Pages Shell

Create semantic navigation, visible keyboard focus and a labelled customer-contact entry point.

Folder: activities/activity-03-accessible-razor-shell

Vibe script

Update the Razor layout and add Pages/Contacts/Index.cshtml. Use native header/nav/main/footer landmarks, one h1, descriptive link text and visible :focus-visible styles. No clickable divs, positive tabindex or ARIA when native HTML suffices.

Step-by-step

Step 1 — Run the baseline

Start LocalCRM and inspect the current DOM and accessibility tree in browser developer tools.

```
dotnet run --project LocalCRM --urls http://localhost:5050
```

Step 2 — Submit the accessibility prompt

Ask for a patch limited to _Layout.cshtml, site.css and the new Contacts page; review the proposed landmarks.

Step 3 — Create the Contacts page

Add a page with one h1 and descriptive navigation.

Create or update LocalCRM/Pages/Contacts/Index.cshtml:

```
@page
@model LocalCRM.Pages.Contacts.IndexModel
@{ ViewData["Title"] = "CRM contacts"; }
<h1>CRM contacts</h1>
<p>Review fictional training contacts.</p>
<a class="btn btn-primary" asp-page="./Create">Add a contact</a>
```

Step 4 — Add its PageModel

Keep the initial GET handler intentionally empty.

Create or update LocalCRM/Pages/Contacts/Index.cshtml.cs:

```
using Microsoft.AspNetCore.Mvc.RazorPages;
namespace LocalCRM.Pages.Contacts;
public sealed class IndexModel : PageModel
{
    public void OnGet() { }
}
```

Step 5 — Restore focus visibility

Add an explicit projector-visible keyboard focus token.

Create or update LocalCRM/wwwroot/css/site.css excerpt:

```
a:focus-visible, button:focus-visible, input:focus-visible, select:focus-visible {
  outline: 3px solid #1f6feb;
  outline-offset: 3px;
}
```

Step 6 — Build and navigate

Compile, open /Contacts, and verify the generated link target.

```
dotnet build LocalCRM/LocalCRM.csproj
curl -i http://localhost:5050/Contacts
```

Step 7 — Run the keyboard checklist

Starting at the address bar, use Tab and Shift+Tab only. Record focus order and confirm Enter opens Create.

Expected evidence: All interactive elements receive visible focus in DOM order.

Step 8 — Inspect the accessibility tree

Confirm header/navigation/main/footer landmarks and a single level-one heading; save evidence/activity-03.md.

Run the supplied snapshot

```
cd activities/activity-03-accessible-razor-shell/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5103
```

Activity 4: Audit Accessibility with axe and Manual Evidence

Create a repeatable automated-plus-manual accessibility gate for the running Razor Pages UI.

Folder: activities/activity-04-accessibility-audit

Vibe script

Review the supplied axe findings and the Razor markup. Propose the smallest semantic fix for each confirmed violation. Do not suppress rules. State which outcomes still require manual keyboard or screen-reader checks.

Step-by-step

Step 1 — Install the audit runner

Use a project-local npm package so the audit version is recorded.

```
mkdir -p tools/a11y && cd tools/a11y
npm init -y
npm install --save-dev @axe-core/cli
```

Step 2 — Start LocalCRM

Keep the application running at the stable Activity URL.

```
dotnet run --project LocalCRM --urls http://localhost:5050
```

Step 3 — Run axe

From the repository root, scan the Contacts page and save JSON output.

```
mkdir -p evidence
npx axe http://localhost:5050/Contacts --save evidence/activity-04-axe.json
```

Step 4 — Inspect violations

For every violation, open the exact node and confirm it is a real problem before changing code.

Step 5 — Use the remediation prompt

Provide only the confirmed violation IDs, HTML snippet and allowed files. Reject rule disabling.

Step 6 — Review and apply

Apply the smallest semantic change. Use native labels/buttons before ARIA and inspect git diff.

Step 7 — Retest

Run the same URL and save the fixed report separately.

```
npx axe http://localhost:5050/Contacts --save evidence/activity-04-axe-fixed.json
```

Expected evidence: No critical or serious automated violations.

Step 8 — Complete manual checks

Record keyboard order, visible focus, heading structure, zoom at 200%, and whether link purpose is clear.

Step 9 — Commit evidence

Commit reviewed source and non-sensitive evidence only.

```
git diff --check
git add LocalCRM evidence/activity-04-axe*.json evidence/activity-04-manual.md
git commit -m 'fix: meet accessible UI gate'
```

Run the supplied snapshot

```
cd activities/activity-04-accessibility-audit/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5104
```

Topic 3: Razor Pages Controls, Binding and Persistence

Alignment: LO3 · K3 / A3

page-focused UI → binding and validation → antiforgery → durable state

Razor Page execution model

Mechanism

1. Route — selects a .cshtml page

2. PageModel — is created through DI

3. Handler — OnGet/OnPost executes

4. ModelState — holds conversion and validation

5. Razor — renders HTML response

Exact contract

- .cshtml: markup plus directives and Tag Helpers
- .cshtml.cs: page handlers and input model
- OnGet: safe read path
- OnPost: form mutation path
- IActionResult: redirect, page or status result

```
public class CreateModel(IContactStore store) : PageModel
{ public void OnGet() { }
  public async Task<IActionResult> OnPostAsync() { /*...*/ } }
```

Failure and verification

- Handler not called: method/name convention mismatched
- Null service: DI registration missing
- Double submit: post returned Page instead of PRG redirect
- Logic in view: separation of concerns collapsed
- Verify: GET selects OnGet
- Verify: POST selects OnPostAsync
- Verify: Successful POST redirects to index

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Razor Pages routing conventions

Mechanism

1. Pages root — defines route discovery

2. Folder — contributes URL segment

3. Index.cshtml — maps to folder root

4. @page — makes the file an endpoint

5. Route values — bind named path parameters

Exact contract

- /Pages/Contacts/Index.cshtml: route /Contacts
- @page "{id:int}": typed route constraint
- asp-page: generates page URL
- asp-route-id: adds route value
- Areas: partition larger applications

```
@page "{id:int}"
<a asp-page="./Details" asp-route-id="@item.Id">View</a>
```

Failure and verification

- 404: @page is missing or path is wrong
- Ambiguous link: relative page path resolves elsewhere
- String id: route constraint was omitted
- Hardcoded URL: renaming page breaks navigation
- Verify: LinkGenerator produces /Contacts/42
- Verify: Non-integer id does not match
- Verify: Renaming folder requires no hardcoded href edit

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Service boundary behind the PageModel

Mechanism

1. PageModel — owns HTTP/UI concerns

2. IContactStore — declares use-case operations

3. Implementation — owns data access

4. DbContext — tracks persistence state

5. Database — enforces durable constraints

Exact contract

- Interface: Task<List<CustomerContact>> ListAsync
- Cancellation: accept RequestAborted token
- DTO/ViewModel: limits exposed fields
- Scoped lifetime: one DbContext per request
- No static state: supports concurrent requests

```
public interface IContactStore
{ Task<int> AddAsync(NewContact input, CancellationToken ct);
  Task<IReadOnlyList<ContactRow>> ListAsync(CancellationToken ct); }
```

Failure and verification

- Fat PageModel: SQL and presentation logic are mixed
- Static list: data races and restart loss
- Sync-over-async: .Result can block request threads
- Leaky entity: persistence shape becomes public contract
- Verify: PageModel unit test substitutes IContactStore
- Verify: Cancellation reaches EF Core
- Verify: Concurrent requests do not share mutable list

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Model binding input sources

Mechanism

1. Value providers — inspect route, query and form

2. Binder — converts text to CLR types

3. Input model — receives allowed fields

4. ModelState — records conversion errors

5. Handler — uses input only after validation

Exact contract

- [BindProperty]: binds form property for non-GET by default
- SupportsGet: explicitly allows GET property binding
- TryParse: controls custom conversion
- Prefix: matches nested field names
- Overposting: prevented by dedicated input model

```
[BindProperty] public NewContact Input { get; set; } = new();
public sealed record NewContact(string Summary, string Category);
```

Failure and verification

- Default values: conversion error was ignored
- Overposting: entity was bound directly
- Wrong prefix: form name does not match property path
- GET mutation: SupportsGet was misused for a command
- Verify: Invalid integer creates ModelState error
- Verify: Unexpected form field is ignored
- Verify: Valid form populates only NewContact fields

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

DataAnnotations and server validation

Mechanism

1. Bind — constructs input model

2. Attributes — evaluate required/range/length rules

3. ModelState — collects all failures

4. Handler guard — returns Page on invalid state

5. Store — runs only after validation succeeds

Exact contract

- [Required]: rejects missing/empty value
- [StringLength]: sets bounded text contract

- [Range]: sets numeric limits
- IValidatableObject: supports cross-field rule
- Database: still enforces storage constraints

```
[Required, EmailAddress]
public string Email { get; init; } = string.Empty;
```

Failure and verification

- Client bypass: server guard was omitted
- Mismatched length: UI and database limits differ
- Opaque message: default error lacks recovery guidance
- Duplicate rule: business validation diverges across endpoints
- Verify: 4-character summary is rejected
- Verify: 120-character summary succeeds
- Verify: Store mock receives no call for invalid input

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Tag Helpers generate consistent controls

Mechanism

1. Razor parser — recognises asp-* attributes

2. Metadata — reads model type and validation

3. Tag Helper — generates name/id/value attributes

4. Antiforgery — injects form token

5. Browser — submits names binder expects

Exact contract

- asp-for: binds label/input/message to one expression
- asp-page: generates route-aware URL
- asp-validation-for: renders ModelState error
- form method=post: participates in antiforgery
- partial: reuses reviewed markup

```
<label asp-for="Input.Email"></label>
<input asp-for="Input.Email" />
<span asp-validation-for="Input.Email"></span>
```

Failure and verification

- Manual name drift: binder receives no value
- Duplicate id: labels and errors bind incorrectly
- Raw URL: route refactor breaks link
- Missing partial: validation scripts never load
- Verify: Rendered label for equals input id
- Verify: Invalid POST redisplays message
- Verify: Generated form contains antiforgery token

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Antiforgery request validation

Mechanism

1. GET form — server emits token pair

2. Browser — stores cookie and hidden field

3. POST — submits both tokens

4. Middleware/filter — validates cryptographic match

5. Handler — runs only after validation

Exact contract

- Cookie: bound to browser session context
- Request token: hidden form value/header
- SameSite: reduces cross-site cookie sending
- POST/PUT/DELETE: unsafe methods require protection
- API auth: bearer-token APIs use different CSRF analysis

```
<form method="post"> <!-- FormTagHelper emits token -->
  <button type="submit">Create</button>
</form>
```

Failure and verification

- 400: token pair is absent or invalid
- Token disabled: cross-site form can execute command
- Mixed auth: cookie API assumed bearer semantics
- Cached form: stale token/cookie pair no longer matches
- Verify: POST without token returns 400
- Verify: Normal browser POST succeeds
- Verify: Unsafe cookie-auth endpoints are inventoried

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

EF Core SQLite persistence

Mechanism

1. Page/API — calls store abstraction

2. DbContext — tracks entity changes

3. Provider — translates LINQ to SQL

4. SQLite — executes transaction

5. SaveChanges — returns affected rows/identity

Exact contract

- DbSet: query and mutation entry point
- Migration: versioned schema change
- AsNoTracking: read-only query optimisation
- Async: does not block while I/O waits

- Constraint: database protects required/unique rules

```
builder.Services.AddDbContext<CrmDbContext>(o =>
  o.UseSqlite(builder.Configuration.GetConnectionString("LocalCRM")));
```

Failure and verification

- No database: migration was never applied
- Locked: long SQLite transaction blocks writer
- N+1: navigation query issues repeated SQL
- Schema drift: manual DB change bypassed migration
- Verify: dotnet ef database update succeeds
- Verify: Create survives application restart
- Verify: Integration test uses an isolated database

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Activity 5: Create and List CRM Contacts with Razor Pages

Implement page-focused create/list handlers through an injectable contact store.

Folder: activities/activity-05-razor-create-list

Vibe script

Implement REQ-001 and REQ-002 with Razor Pages. Introduce IContactStore and an in-memory implementation registered as singleton for this temporary Activity only. Use a dedicated NewContact input model and Post/Redirect/Get. Limit changes to Models, Services, Pages/Contacts and Program.cs.

Step-by-step

Step 1 — Create the domain/input types

Use separate input and output shapes to prevent overposting.

Create or update LocalCRM/Models/CustomerContact.cs:

```
namespace LocalCRM.Models;
public sealed record CustomerContact(int Id, string Name, string Email, string Stage,
DateTimeOffset CreatedAt);
public sealed record NewContact(string Name, string Email, string Stage);
```

Step 2 — Declare the service boundary

Keep HTTP details out of the store contract.

Create or update LocalCRM/Services/IContactStore.cs:

```
using LocalCRM.Models;
namespace LocalCRM.Services;
public interface IContactStore
{
 Task<int> AddAsync(NewContact input, CancellationToken ct);
 Task<IReadOnlyList<CustomerContact>> ListAsync(CancellationToken ct);
}
```

Step 3 — Implement temporary memory storage

Use locking/concurrent collections or a reviewed implementation; make restart loss explicit.

Step 4 — Register the service

Add the service before builder.Build().

Create or update Program.cs registration:

```
builder.Services.AddSingleton<IContactStore, InMemoryContactStore>();
```

Step 5 — Build the Create PageModel

Bind a dedicated input model, guard ModelState, call the store and redirect to Index.

Step 6 — Build the form markup

Use asp-for labels, inputs and validation messages. Keep the Form Tag Helper enabled for antiforgery.

Step 7 — Load the list

Inject IContactStore into IndexModel and call ListAsync(HttpContext.RequestAborted).

Step 8 — Verify PRG

Submit a valid contact, observe 302 to /Contacts, then refresh.

```
curl -i -c cookies.txt -b cookies.txt http://localhost:5050/Contacts/Create
```

Expected evidence: Browser submission redirects; refresh does not create a duplicate.

Step 9 — Verify restart limitation

Restart the app and confirm the temporary list is empty; record why Activity 8 replaces it.

Run the supplied snapshot

```
cd activities/activity-05-razor-create-list/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5105
```

Activity 6: Enforce Validation and Antiforgery Controls

Reject invalid or forged commands while returning accessible recovery guidance.

Folder: activities/activity-06-validation-and-antiforgery

Vibe script

> Harden the Create Razor Page. Name is required (2–80 characters), Email is required and valid, and Stage is Lead, Qualified or Customer. Add accessible field errors and validation summary. Preserve automatic antiforgery and add tests for invalid input and missing token.

Step-by-step

Step 1 — Add validated input model

Use DataAnnotations on a mutable page input type.

Create or update LocalCRM/Models/CreateContactInput.cs:

```
using System.ComponentModel.DataAnnotations;
namespace LocalCRM.Models;
public sealed class CreateContactInput
{
 [Required, StringLength(80, MinimumLength = 2)] public string Name { get; set; } =
string.Empty;
 [Required, EmailAddress] public string Email { get; set; } = string.Empty;
 [Required] public string Stage { get; set; } = "Lead";
}
```

Step 2 — Guard the handler

Return Page() before calling the store when ModelState is invalid.

Create or update OnPost excerpt:

```
if (!ModelState.IsValid) return Page();
await store.AddAsync(new(Input.Name, Input.Email, Input.Stage), HttpContext.RequestAborted);
return RedirectToPage("./Index");
```

Step 3 — Render errors accessibly

Add asp-validation-summary and asp-validation-for. Ensure visible text tells the learner how to repair the field.

Step 4 — Build and submit invalid values

Try an empty name, a one-character name and a malformed email address.

Expected evidence: Each invalid request redisplays the form and does not call the store.

Step 5 — Inspect the antiforgery token

Open page source and locate __RequestVerificationToken plus the matching cookie.

Step 6 — Prove token enforcement

Send a raw POST without a token.

```
curl -i -X POST http://localhost:5050/Contacts/Create -d
'Input.Name=Aisha&Input.Email=aisha@example.test&Input.Stage=Lead'
```

Expected evidence: HTTP 400; the handler does not persist a contact.

Step 7 — Prove the normal path

Submit via the browser with a valid token and confirm redirect plus stored item.

Step 8 — Run accessibility regression

Run axe and keyboard checks on both initial and error states.

Run the supplied snapshot

```
cd activities/activity-06-validation-and-antiforgery/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5106
```

Topic 4: Controller APIs and Component Interoperability

Alignment: LO4 · K4 / A4

resource contract → controller boundary → persistence → observable HTTP evidence

Resource-oriented HTTP semantics

Mechanism

1. Client intent — chooses safe/idempotent operation

2. Route — names resource collection/item

3. Controller — maps verb to action

4. Store — reads or changes state

5. Representation — returns status, headers and JSON

Exact contract

- GET /api/contacts: list; safe and idempotent

- POST /api/contacts: create; not inherently idempotent
- GET /api/contacts/{id}: read one
- PUT /api/contacts/{id}: replace; idempotent
- DELETE /api/contacts/{id}: remove; idempotent outcome

```
[ApiController]
[Route("api/contacts")]
public sealed class ContactsController : ControllerBase { }
```

Failure and verification

- Verb tunnel: all commands use POST without semantics
- Noun mismatch: route describes implementation, not resource
- GET mutates: caches/retries can change state
- No Location: client cannot find created resource
- Verify: OpenAPI lists five intended operations
- Verify: GET never changes row count
- Verify: POST returns 201 and Location header

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Controller routing and action selection

Mechanism

1. Endpoint routing — matches controller route tokens

2. Constraint — filters by verb and parameter type

3. Action descriptor — selects one candidate

4. Model binding — supplies arguments

5. Action — returns ActionResult

Exact contract

- [Route]: controller-level path prefix
- [HttpGet("{id:int}")]: verb plus constrained template
- [FromRoute]: explicit path source
- CreatedAtAction: generates canonical item URL
- Name: supports stable link generation

```
[HttpGet("{id:int}", Name = "GetContact")]
public async Task<ActionResult<ContactDto>> Get(int id) { /*...*/ }
```

Failure and verification

- AmbiguousMatch: two actions share equivalent route
- 404: constraint rejects supplied id
- Wrong URL: CreatedAtAction route values do not match
- Leaked action: public method lacks NonAction/verb attribute
- Verify: Route table has no duplicate method+path
- Verify: id=abc produces 404
- Verify: Location URL resolves to the new representation

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

DTO and domain boundary

Mechanism

1. JSON — deserialises into request DTO

2. Validation — checks transport contract

3. Mapping — creates domain command/entity

4. Store — persists allowed fields

5. Response DTO — returns stable public shape

Exact contract

- CreateContactDto: client-settable fields only
- ContactDto: id/status/timestamps safe to expose
- Entity: persistence and domain state
- Mapping: explicit code makes changes reviewable
- Versioning: contract change is deliberate

```
public sealed record CreateContactDto(
  [Required, StringLength(80)] string Name,
  [Required, EmailAddress] string Email, string Stage);
```

Failure and verification

- Mass assignment: entity is accepted directly
- Circular JSON: navigation properties leak
- Contract drift: database rename breaks clients
- Sensitive field: internal note appears in response
- Verify: Unknown JSON property is handled by policy
- Verify: Response excludes internal fields
- Verify: Mapping test covers every public property

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

API model binding and automatic 400

Mechanism

1. Input formatter — reads application/json

2. Deserializer — constructs DTO

3. Validation — evaluates DataAnnotations

4. ApiController — short-circuits invalid ModelState

5. ProblemDetails — returns machine-readable error

Exact contract

- Content-Type: must select JSON input formatter
- 400: syntactically valid but contract-invalid

- 415: unsupported media type
- ValidationProblemDetails: errors keyed by field
- Nullable: requiredness should be explicit

```
curl -i -X POST /api/contacts \
+ -H "Content-Type: application/json" -d "{\"summary\":\"\"}"
```

Failure and verification

- 400 hidden: client ignores ProblemDetails body
- 415: Content-Type header is missing/wrong
- Empty object: nullable/default contract too permissive
- Exception: custom binder throws instead of reporting error
- Verify: Invalid DTO returns application/problem+json
- Verify: Error dictionary names summary
- Verify: Controller action is not invoked

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

CRUD status and concurrency contract

Mechanism

1. Read version — client receives current ETag/version

2. Edit — client sends If-Match/version

3. Store — updates only matching version

4. Conflict — zero rows signals stale write

5. Recover — client reloads and resolves

Exact contract

- 200: successful read/update with body
- 201: created with Location
- 204: successful delete without body
- 404: resource absent
- 409/412: state conflict/precondition failed

```
if (dto.Version != entity.Version)
  return Conflict(new ProblemDetails { Title = "Stale contact" });
```

Failure and verification

- Lost update: last writer silently overwrites
- Wrong 200: missing resource returns success
- Duplicate create: retry creates second record
- 204 with body: response violates its status semantics
- Verify: Stale update produces 409/412
- Verify: Delete missing id produces 404
- Verify: Create provides resolvable Location

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Async EF Core API path

Mechanism

1. Controller — accepts RequestAborted

2. LINQ — builds IQueryable

3. Provider — translates expression to SQL

4. Database — performs asynchronous I/O

5. DTO projection — materialises only required columns

Exact contract

- ToListAsync: executes query asynchronously
- Select: projects before materialisation
- AsNoTracking: avoids change tracker for reads
- CancellationToken: stops abandoned work
- SaveChangesAsync: persists transaction

```
var rows = await db.Contacts.AsNoTracking()
 .Select(x => new ContactDto(x.Id, x.Name, x.Email, x.Stage))
 .ToListAsync(ct);
```

Failure and verification

- Client eval: expression cannot translate
- Over-fetch: entity graph is loaded unnecessarily
- No cancellation: work continues after disconnect
- Parallel DbContext: same context is used concurrently
- Verify: SQL log selects only DTO columns
- Verify: Cancelled request ends query
- Verify: Load test shows no sync blocking hotspot

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

OpenAPI as interoperability contract

Mechanism

1. Endpoints — expose route and metadata

2. Generator — builds OpenAPI document

3. Schema — describes DTO types and validation

4. Client/tool — generates calls or UI

5. Contract test — detects breaking change

Exact contract

- paths: method and route operations
- operationId: stable generated-client method
- components/schemas: reusable JSON shapes
- responses: document every intended status

- securitySchemes: declares authentication mechanism

```
builder.Services.AddOpenApi();
if (app.Environment.IsDevelopment()) app.MapOpenApi();
```

Failure and verification

- Undocumented status: client cannot model failure
- Schema leak: entity exposes internal fields
- Breaking rename: generated client no longer compiles
- Prod exposure: interactive docs reveal attack surface
- Verify: openapi.json validates
- Verify: Snapshot diff contains no accidental breaking change
- Verify: 201/400/404 schemas are declared

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Integration tests and ProblemDetails

Mechanism

1. WebApplicationFactory — starts in-memory test host

2. HttpClient — sends real HTTP request

3. Pipeline — routing/binding/filters execute

4. Test store — isolates deterministic data

5. Assertion — checks status, headers and JSON

Exact contract

- Arrange: seed isolated database
- Act: call public endpoint only
- Assert: status + content type + shape
- ProblemDetails: stable error envelope
- Trace id: links client failure to server log

```
var response = await client.PostAsJsonAsync("/api/contacts", new { summary = "" });
Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
```

Failure and verification

- Unit-only: middleware/routing regression is invisible
- Shared DB: tests affect each other
- Body-only: wrong status still passes
- Stack trace leak: error response exposes internals
- Verify: Happy and invalid paths run through HTTP
- Verify: ProblemDetails contains traceId
- Verify: Tests pass in parallel with isolated state

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Activity 7: Expose a Controller-Based CRUD API

Add a JSON controller surface that reuses the application service without exposing persistence entities.

Folder: activities/activity-07-controller-api

Vibe script

> Add an ApiController at /api/contacts that supports GET collection, GET item and POST create. Use transport DTOs, IContactStore, CreatedAtAction and CancellationToken. Do not return persistence entities directly. Include 400 and 404 evidence.

Step-by-step

Step 1 — Enable controllers

Register controller services and map controller endpoints.

Create or update Program.cs additions:

```
builder.Services.AddControllers();
// after middleware
app.MapControllers();
```

Step 2 — Create DTOs

Separate client input from response shape.

Create or update LocalCRM/Contracts/ContactDtos.cs:

```
using System.ComponentModel.DataAnnotations;
namespace LocalCRM.Contracts;
public sealed record CreateContactDto([Required, StringLength(80, MinimumLength=2)] string
Name, [Required, EmailAddress] string Email, [Required] string Stage);
public sealed record ContactDto(int Id, string Name, string Email, string Stage,
DateTimeOffset CreatedAt);
```

Step 3 — Extend the service

Add GetAsync(int, CancellationToken) to IContactStore and both implementations.

Step 4 — Create ContactsController

Use [ApiController], [Route], [HttpGet] and [HttpPost]; map entity to DTO explicitly.

Step 5 — Return 201 correctly

Use CreatedAtAction(nameof(Get), new { id }, dto) so the Location header is generated from routing.

Step 6 — Probe list

Start the app and request the collection.

```
curl -i http://localhost:5050/api/contacts
```

Expected evidence: HTTP 200 and a JSON array.

Step 7 — Probe create

Send a valid JSON command.

```
curl -i -X POST http://localhost:5050/api/contacts -H 'Content-Type: application/json' -d
'{"name":"Deepa Nair","email":"deepa@example.test","stage":"Lead"}'
```

Expected evidence: HTTP 201, JSON body and Location header.

Step 8 — Probe failures

Send an invalid DTO and request an absent id.

```
curl -i -X POST http://localhost:5050/api/contacts -H 'Content-Type: application/json' -d
'{"name":"","email":"not-email","stage":"Lead"}'
curl -i http://localhost:5050/api/contacts/9999
```

Expected evidence: HTTP 400 ProblemDetails, then HTTP 404.

Run the supplied snapshot

```
cd activities/activity-07-controller-api/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5107
```

Activity 8: Prove Persistence and Interoperability

Replace volatile state with EF Core SQLite, publish an OpenAPI contract and verify components through integration tests.

Folder: activities/activity-08-sqlite-openapi-tests

Vibe script

> Replace InMemoryContactStore with EF Core SQLite behind IContactStore. Add a migration, async cancellation-aware queries, OpenAPI in Development, and WebApplicationFactory tests for 201, 400 and 404. Preserve existing page and API contracts.

Step-by-step

Step 1 — Add reviewed packages

Pin compatible .NET 10 package versions rather than accepting an AI-chosen floating version.

```
dotnet add LocalCRM package Microsoft.EntityFrameworkCore.Sqlite --version 10.0.12
dotnet add LocalCRM package Microsoft.EntityFrameworkCore.Design --version 10.0.12
dotnet add LocalCRM package Microsoft.AspNetCore.OpenApi --version 10.0.12
```

Step 2 — Create CrmDbContext

Define ContactEntity with database key and bounded required columns.

Step 3 — Configure connection

Keep only the local non-secret SQLite path in Development settings.

Create or update LocalCRM/appsettings.Development.json excerpt:

```
{ "ConnectionStrings": { "LocalCRM": "Data Source=localcrm.db" } }
```

Step 4 — Register EF Core

Use AddDbContext and a scoped SqliteContactStore.

Step 5 — Create/apply migration

Install or invoke dotnet-ef, inspect generated migration, then apply it.

```
dotnet tool install --global dotnet-ef --version 10.0.12
dotnet ef migrations add InitialCreate --project LocalCRM
dotnet ef database update --project LocalCRM
```

Step 6 — Enable OpenAPI

Register AddOpenApi and map it only in Development.

Step 7 — Capture the contract

Run the app and save the generated document.

```
curl -fsS http://localhost:5050/openapi/v1.json -o evidence/openapi-v1.json
```

Expected evidence: Valid JSON with /api/contacts operations and DTO schemas.

Step 8 — Add integration test project

Create xUnit project, reference LocalCRM and add Microsoft.AspNetCore.Mvc.Testing.

Step 9 — Test public HTTP behaviour

Use WebApplicationFactory with an isolated SQLite database. Assert status, Location, media type and ProblemDetails shape.

Step 10 — Verify persistence

Create through API, restart the app and GET the same id.

Expected evidence: The row remains after restart and both UI/API read the same store.

Run the supplied snapshot

```
cd activities/activity-08-sqlite-openapi-tests/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5108
```

Topic 5: Production Hosting, Azure and Design Evidence

Alignment: LO5 · K5 / A5

deployable artifact → observable service → safe release → traceable design record

Environment-specific configuration and secrets

Mechanism

1. Source — contains safe defaults and option schema

2. Build — produces environment-neutral artifact

3. Platform — injects settings and secret references

4. Options — validates configuration on startup

5. Runtime — logs source names, never secret values

Exact contract

- ASPNETCORE_ENVIRONMENT: Development/Staging/Production
- App__FeatureFlag: environment variable key encoding
- Key Vault reference: secret stays outside app settings
- ValidateOnStart: fails before traffic
- Rotation: changes secret without source edit

```
var cs = builder.Configuration.GetConnectionString("LocalCRM")
  ?? throw new InvalidOperationException("Missing LocalCRM connection");
```

Failure and verification

- Secret in repo: credential entered appsettings.json
- Wrong slot value: setting was not marked deployment-slot specific
- Silent default: production uses developer fallback
- Log leak: configuration object was serialised
- Verify: Secret scan is clean
- Verify: Production start fails without required setting
- Verify: Rotated value is picked up via platform process

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Structured logging and health checks

Mechanism

1. Request — creates trace identifier

2. Logger — emits template plus named properties

3. Provider — ships events to sink

4. Health check — probes dependency readiness

5. Operator — correlates failure by trace

Exact contract

- Template: Contact {ContactId} created
- Level: Information/Warning/Error chosen by action
- Liveness: process is running
- Readiness: dependencies can serve traffic
- PII: not emitted as log property

```
builder.Services.AddHealthChecks().AddDbContextCheck<CrmDbContext>();
app.MapHealthChecks("/health/ready");
```

Failure and verification

- String log: fields cannot be queried
- False green: liveness ignores failed database
- PII leak: request body is logged
- No trace: client error cannot be correlated
- Verify: /health/live and /health/ready have distinct meaning
- Verify: 500 log carries trace id
- Verify: No sensitive payload appears in sink

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Publish output and hosting boundary

Mechanism

1. dotnet publish — compiles release artifact

2. Artifact — contains assemblies, deps and config

3. Host — starts process with environment

4. Kestrel — listens on bound address

5. Proxy — terminates TLS and forwards headers

Exact contract

- Framework-dependent: small artifact requires installed runtime
- Self-contained: larger artifact bundles runtime

- ForwardedHeaders: trust only known proxy
- PORT/URLS: platform supplies binding
- Shutdown: SIGTERM allows graceful stop

```
dotnet publish -c Release -o ./publish
ASPNETCORE_URLS=http://+:8080 dotnet ./publish/LocalCRM.dll
```

Failure and verification

- 502: process crashed or bound wrong port
- Redirect loop: proxy headers are not handled
- Runtime missing: framework-dependent target lacks .NET
- Partial artifact: publish folder was copied inconsistently
- Verify: Release artifact starts in a clean directory
- Verify: HTTP probe reaches bound port
- Verify: SIGTERM stops without dropping active request

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Azure App Service component model

Mechanism

1. Resource group — owns lifecycle boundary

2. App Service plan — provides compute and scale unit

3. Web app — hosts the deployment

4. Configuration — injects app settings and identity

5. Monitoring — collects logs, metrics and health

Exact contract

- Region: latency/data residency decision
- Plan SKU: cost, scale and feature envelope
- Linux/Windows: runtime hosting choice
- Always On: avoids idle unload where supported
- Health check: removes unhealthy instances

```
az webapp config appsettings set --resource-group rg-localcrm \
+ --name localcrm-web --settings ASPNETCORE_ENVIRONMENT=Production
```

Failure and verification

- Wrong plan: required feature is unavailable
- Cold/idle: Always On not enabled where needed
- Unhealthy instance: health path not configured
- Configuration drift: portal-only edits bypass design record
- Verify: Resource inventory names RG, plan and app
- Verify: Health check path is configured
- Verify: Deployment settings match documented environment

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Deployment slots and safe promotion

Mechanism

1. Deploy staging — new build receives no production traffic

2. Warm/probe — health and smoke tests execute

3. Swap — hostnames move between slots

4. Observe — errors and latency are compared

5. Rollback — swap back if acceptance fails

Exact contract

- Slot setting: secret/connection stays with slot
- Warm-up: application initialises before swap
- Preview: validates configuration changes
- Smoke test: uses public contract
- Rollback: predefined threshold and command

```
az webapp deployment slot swap -g rg-localcrm -n localcrm-web \
+ --slot staging --target-slot production
```

Failure and verification

- Bad swap: slot-specific setting was not sticky
- Warm-up miss: first production request compiles/initialises
- No rollback: failed build remains live
- Schema incompatibility: old app cannot run after migration
- Verify: Staging smoke test passes before swap
- Verify: Production health remains green
- Verify: Rollback step is rehearsed and timed

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Managed identity and database access

Mechanism

1. Web app — obtains platform-managed identity

2. Token credential — requests Entra token

3. Azure SQL — authorises identity principal

4. EF Core — opens connection without password

5. Audit — records principal and query outcome

Exact contract

- System assigned: identity lifecycle follows app
- User assigned: identity can be shared/reused
- Least privilege: grant only required database role
- DefaultAzureCredential: local and hosted credential chain
- No password: rotation burden moves to platform

```
builder.Services.AddAzureClients(c => c.UseCredential(new DefaultAzureCredential()));
```

Failure and verification

- Login failed: identity lacks database user/role
- Wrong tenant: credential resolved unexpected account
- Overprivilege: app identity is database owner
- Local mismatch: developer chain differs from hosted identity
- Verify: App connects with passwordless identity
- Verify: Database audit names the app principal
- Verify: Denied operation proves least privilege

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

CI/CD gates and smoke evidence

Mechanism

1. Commit — starts immutable build

2. Restore/build — recreates dependencies and binaries

3. Test/scan — blocks defects and secrets

4. Deploy staging — releases exact artifact

5. Smoke/promote — verifies contract before production

Exact contract

- Artifact SHA: links deployed bytes to commit
- dotnet test: unit and integration gate
- Dependency scan: known-vulnerability control
- Environment approval: separates build from promotion
- Smoke: health plus one business transaction

```
dotnet restore --locked-mode
dotnet build -c Release --no-restore
dotnet test -c Release --no-build
```

Failure and verification

- Mutable deploy: server rebuilds from source
- No lock: dependency version changes silently
- Green build/bad app: no HTTP smoke test
- Untraceable: artifact SHA not recorded
- Verify: Pipeline recreates build from lock file
- Verify: Artifact hash is recorded
- Verify: Smoke test proves health and create/read path

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

ADR, C4 and requirements traceability

Mechanism

1. Requirement — states user-visible need and acceptance

2. C4 view — shows people, systems and containers

3. ADR — records decision, context and consequences

4. Interface contract — OpenAPI/schema defines boundary

5. Evidence — tests and release link close trace

Exact contract

- Requirement ID: REQ-001 stable identifier
- Decision: ADR-0001 accepted/rejected options
- Component: owner and responsibility
- Interface: method/path/schema/status
- Verification: test ID, result and build SHA

```
# ADR-0001: Razor Pages + controller API
Status: Accepted
Decision: page-focused UI; JSON API boundary
Consequences: shared service; two HTTP surfaces.
```

Failure and verification

- Diagram only: no decision rationale or consequences
- Orphan test: evidence maps to no requirement
- Stale contract: OpenAPI differs from deployed endpoint
- Missing owner: component boundary cannot be governed
- Verify: Every LO requirement maps to component and test
- Verify: ADR names at least one rejected option
- Verify: OpenAPI hash matches released build

Source basis: Microsoft Learn — ASP.NET Core 10.0. Reviewed for the .NET 10 course baseline on 12 September 2026.

Activity 9: Prepare an Azure-Ready Release

Create a production-neutral publish artifact with validated configuration, health endpoints and smoke evidence.

Folder: activities/activity-09-production-readiness

Vibe script

Prepare LocalCRM for Azure App Service without performing a live deployment. Add validated typed options, /health/live and /health/ready, structured startup logging, and a bash smoke script. Do not add credentials, an Azure resource or a portal-only step.

Step-by-step

Step 1 — Define the configuration contract

Add AppOptions with required SupportEmail and public feature flags; bind and ValidateOnStart.

Step 2 — Add health checks

Map liveness and database readiness separately so operators can distinguish process from dependency health.

Step 3 — Review production settings

Create docs/deployment-configuration.md naming each key, owner, source and slot-sticky decision. Never place a live value in the table.

Step 4 — Publish Release

Build an environment-neutral artifact.

```
dotnet publish LocalCRM/LocalCRM.csproj -c Release -o publish
```

Step 5 — Inspect the artifact

Confirm assemblies and static content exist, while source, .git and local database do not.

```
find publish -maxdepth 2 -type f | sort
find publish -name '*.db' -o -name '.git'
```

Step 6 — Run from publish only

Start the compiled DLL on port 8080 with a safe required setting.

```
App__SupportEmail=support@example.test ASPNETCORE_URLS=http://localhost:8080 dotnet
publish/LocalCRM.dll
```

Step 7 — Create smoke-test script

Probe liveness, readiness, list and one invalid create. Make the script exit non-zero on a mismatch.

Step 8 — Capture evidence

Run the smoke script and record build SHA, publish file hash and observed statuses.

```
sh activities/activity-09-production-readiness/assets/smoke.sh http://localhost:8080
shasum -a 256 publish/LocalCRM.dll
```

Step 9 — Model Azure resources

Document resource group, App Service plan, web app, settings, identity, database and monitoring boundaries; do not claim deployment occurred.

Run the supplied snapshot

```
cd activities/activity-09-production-readiness/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5109
```

Activity 10: Produce the Traceable Design and Release Record

Map user specifications to components, interface contracts, decisions and verification evidence.

Folder: activities/activity-10-design-documentation

Vibe script

> Using requirements.md, component-map.md, openapi-v1.json and the Activity evidence, draft ADR-0001 and a traceability matrix. Every requirement must map to a component, interface or page, control, test/evidence and status. Mark gaps; do not invent evidence.

Step-by-step

Step 1 — Inventory authoritative inputs

List requirements, code components, OpenAPI contract, tests and evidence. Mark missing files before generating documentation.

Step 2 — Create a system context diagram

Show Staff User, LocalCRM, Azure App Service, database and monitoring as named trust/ownership boundaries.

Step 3 — Create a container diagram

Show Razor Pages UI, controller API, application service, EF Core adapter and SQLite/Azure SQL boundary.

Step 4 — Write ADR-0001

Record context, decision, at least two considered options, consequences and status. Use exact repository paths.

Step 5 — Build the traceability matrix

Use columns Requirement, Design component, Interface/control, Verification evidence, Status and Owner.

Step 6 — Hash the interface contract

Record the OpenAPI file hash beside the build SHA.

```
shasum -a 256 evidence/openapi-v1.json
git rev-parse HEAD
```

Step 7 — Close evidence gaps

Run only the missing checks. Never mark a row Pass from an AI claim or expected result alone.

Step 8 — Review with the stakeholder

Ask a peer to confirm the five LOs and REQ-001–004 are represented. Record reviewer/date and unresolved actions.

Step 9 — Package the release record

Create docs/release-v6.1.md listing artifact hash, configuration assumptions, known limitations, rollback and approval state.

Step 10 — Final gate

Run format, build, tests, secret scan, OpenAPI check and smoke test; attach command output to the release record.

```
dotnet format LocalCRM/LocalCRM.csproj --verify-no-changes
dotnet build LocalCRM/LocalCRM.csproj -c Release
dotnet test -c Release
git status --short
```

Expected evidence: All checks pass and the working tree contains only intentional documentation/evidence changes.

Run the supplied snapshot

```
cd activities/activity-10-design-documentation/src/LocalCRM
dotnet restore
dotnet run --urls http://localhost:5110
```

Quick Command Reference

Command	Purpose
dotnet --info	Record the selected SDK/runtime.
dotnet restore	Restore declared packages.
dotnet build -c Release	Compile the reviewed source.
dotnet test	Run the automated evidence suite.

Command	Purpose
dotnet run --urls http://localhost:PORT	Start on a known local endpoint.
curl -i URL	Inspect status, headers and body.
git diff --check	Detect whitespace errors in the proposed patch.
git diff --name-only	Confirm the AI stayed inside scope.

Assessment Flow

1. Scan the TRAQOM QR code on the LMS.

2. Complete Assessment Digital Attendance.

3. Complete the Written Assessment, then Practical Performance (open book).

4. Submit assessment answers and evidence on the LMS.

5. Sign the Assessment Summary Record.

LMS: https://lms-tms.tertiaryinfotech.com/

Support

Tertiary Infotech Academy Pte Ltd · enquiry@tertiaryinfotech.com · +65 6100 0613 · https://www.tertiarycourses.com.sg